ANNA UNIVERSITY

MADRAS INSTITUTE OF TECHNOLOGY, CHENNAI – 600 044

Department of Electronics and Communication Engineering

EC23020 – PIC MICROCONTROLLERS

MINI PROJECT REPORT

VEHICLE-TO-VEHICLE (V2V) COMMUNICATION SYSTEM

Two-way wireless exchange of speed and motion data using nRF24L01 and MPU6500

TEAM MEMBERS

S.No	Name	Register Number
1	VASANTHARAJ G	2024504052
2	SANMUGAVEL B	2024504547
3	GOKUL ANAND A	2024504702

1. ABSTRACT

Many road accidents happen because a driver cannot see what the vehicle ahead or around a blind bend is doing. Vehicle-to-Vehicle (V2V) communication lets vehicles share their own state directly by radio, without any tower or internet. In this project two vehicle nodes – an Arduino Mega 2560 (Vehicle 1) and an Arduino Nano (Vehicle 2) – each carry an nRF24L01 2.4 GHz transceiver and an MPU-series accelerometer/gyroscope. Every second, Vehicle 1 transmits a packet with its ID, packet counter, speed and 3-axis acceleration and rotation; Vehicle 2 receives it and immediately replies with its own packet. Both nodes print the exchanged data on the serial monitor, which is also read by a browser dashboard. The design uses the three on-chip serial buses studied in this course – SPI, I2C and UART – exactly as on the PIC 16F877 (Units I and V).

2. OBJECTIVES

- **Wireless link:** establish a reliable two-way radio link between two microcontroller nodes.
- **Sensing:** read real-time acceleration and angular rate from an MPU sensor over I2C.
- **Data exchange:** pack ID, counter, speed and motion data into one fixed 20-byte structure.
- **Fault tolerance:** keep the radio link running even if a sensor is missing or a read fails.

3. HARDWARE COMPONENTS

Component	Qty	Function in the project
Arduino Mega 2560 (ATmega2560)	1	Vehicle 1 controller – starts every exchange, waits 250 ms for reply
Arduino Nano (ATmega328P)	1	Vehicle 2 controller – listens, then answers each packet
nRF24L01 2.4 GHz transceiver	2	Wireless link between the vehicles (SPI interface, 3.3 V)
MPU6500 / MPU6050 IMU	2	3-axis accelerometer (±2 g) + 3-axis gyroscope (±250 °/s), I2C
USB cable + PC / laptop	2	Power, programming and UART serial monitor / dashboard
Jumper wires, breadboard	–	Connections between the modules

4. SYSTEM BLOCK DIAGRAM

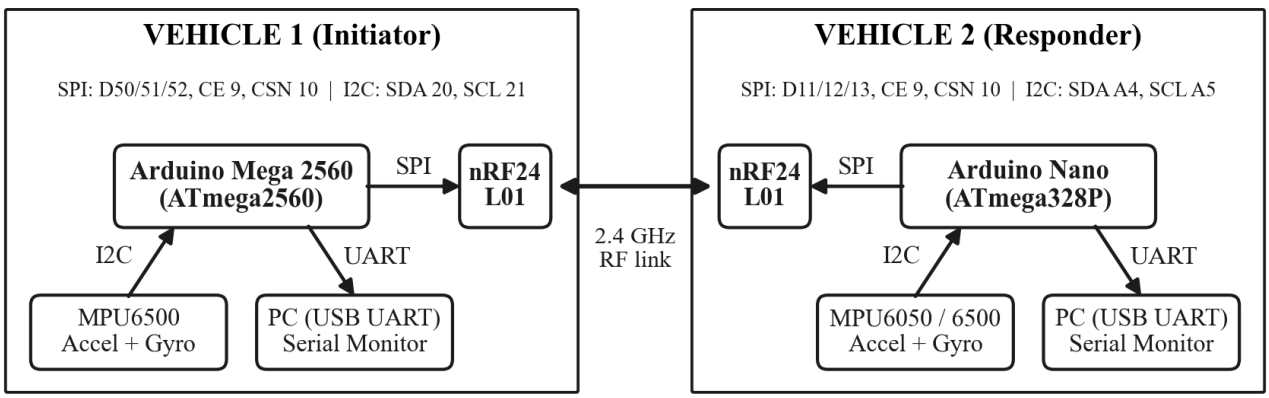


Fig. 1 Block diagram of the two-vehicle V2V node setup

5. COMMUNICATION PROTOCOLS USED

Protocol	Link	How it is used in this project
SPI (Serial Peripheral Interface)	MCU ↔ nRF24L01	Full-duplex, 4-wire bus (MOSI, MISO, SCK, CSN) with the MCU as master, Mode 0. CSN (pin 10) selects the radio, CE (pin 9) switches it between TX and RX. All radio registers and the payload FIFO are written and read over SPI.
I2C (Inter-Integrated Circuit)	MCU ↔ MPU sensor	2-wire bus (SDA, SCL), 7-bit slave address 0x68. Mega reads WHO_AM_I (0x75), wakes the chip (PWR_MGMT_1 = 0x01) and burst-reads 14 bytes from ACCEL_XOUT_H (0x3B). Nano uses the Adafruit MPU6050 library over the same bus.
RF – Enhanced ShockBurst™ (2.4 GHz ISM)	Vehicle 1 ↔ Vehicle 2	GFSK radio on channel 100 (2.500 GHz) at 250 kbps, PA LOW, pipe address "CAR01". Hardware auto-acknowledgement, CRC check and auto-retransmit – setRetries(5, 15) = up to 15 retries, 1.5 ms apart. Half-duplex: each node switches between stopListening() and startListening().
UART (USB Serial)	MCU ↔ PC	Asynchronous serial at 9600 baud, 8-N-1, through the on-board USB bridge. Shows sent / received data on the Serial Monitor and feeds the Web-Serial V2V dashboard.

Data packet (identical struct on both boards – 20 bytes, below the 32-byte nRF24L01 payload limit):

Field	vehicleID	counter	speed	mpuPresent	ax, ay, az	gx, gy, gz
Type	uint8	uint16	float	uint8	3 × int16	3 × int16
Bytes	1	2	4	1	6	6
Meaning	1 = Mega 2 = Nano	packet no.	km/h (demo)	1 = sensor OK	milli-g	0.1 °/s

6. FLOWCHART

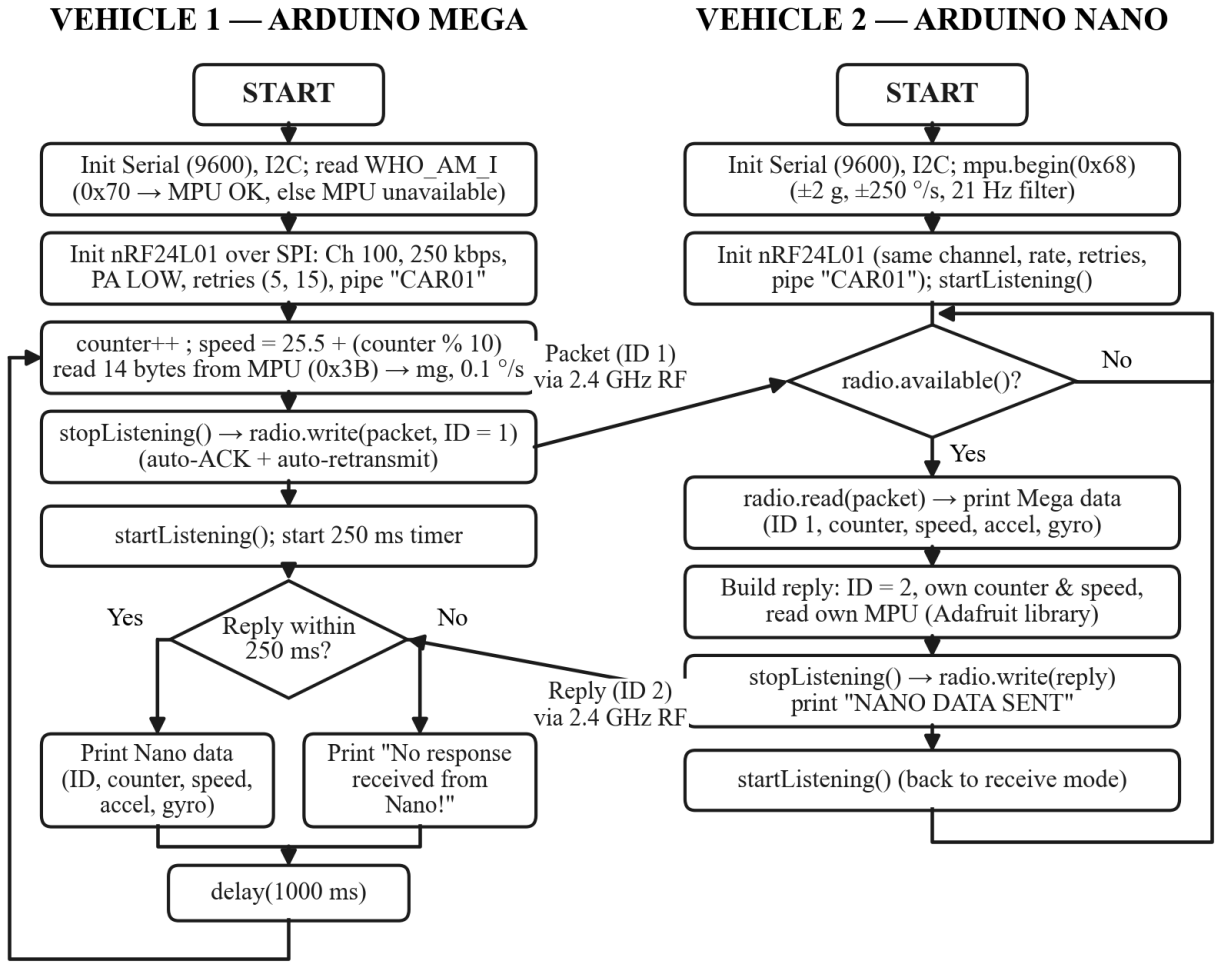


Fig. 2 Program flow of Vehicle 1 (Mega) and Vehicle 2 (Nano)

7. WORKING OF THE PROJECT

- **Initialisation:** Both boards start UART at 9600 baud and the I2C bus. The Mega checks the sensor's WHO_AM_I register (0x70 = MPU6500) and sets ±2 g / ±250 °/s ranges; the Nano starts its MPU through the Adafruit library. Both radios are configured identically over SPI.
- **Sensor reading:** Raw 16-bit values are scaled – acceleration ÷ 16384 LSB/g → milli-g and angular rate ÷ 131 LSB/(°/s) → 0.1 °/s – so they fit in int16 fields without floating-point maths.
- **Transmit (Vehicle 1):** Every 1 s the Mega increments its counter, updates the demo speed (25.5 + counter mod 10), reads the MPU and sends the 20-byte packet. The nRF24L01 automatically waits for an ACK and retransmits if it is lost.
- **Receive and reply (Vehicle 2):** The Nano stays in receive mode. When radio.available() is true it reads and prints the Mega's packet, fills its own packet (ID 2, own counter, speed and MPU data), switches to TX, sends the reply and returns to RX.
- **Response check:** After sending, the Mega listens for 250 ms. If the reply arrives it prints the Nano's data; otherwise it prints "No response received from Nano!" and tries again next cycle.
- **Fault tolerance:** If an MPU is absent or a read fails, mpuPresent is set to 0 and zeros are sent, so the radio link continues. The program halts only if the nRF24L01 itself is not detected.

8. HARDWARE DEMONSTRATION

Photographs of the two vehicle units built for the demonstration are given below.

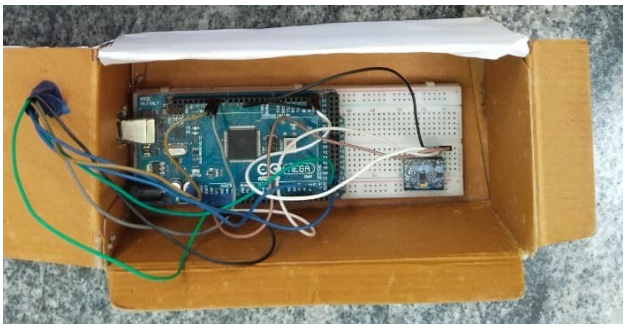


Fig. 3 Vehicle 1 – Arduino Mega + nRF24L01 + MPU6500

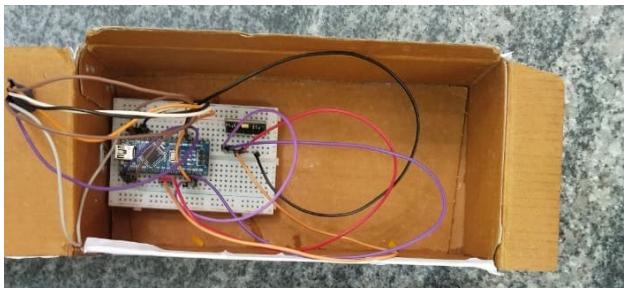


Fig. 4 Vehicle 2 – Arduino Nano + nRF24L01 + MPU

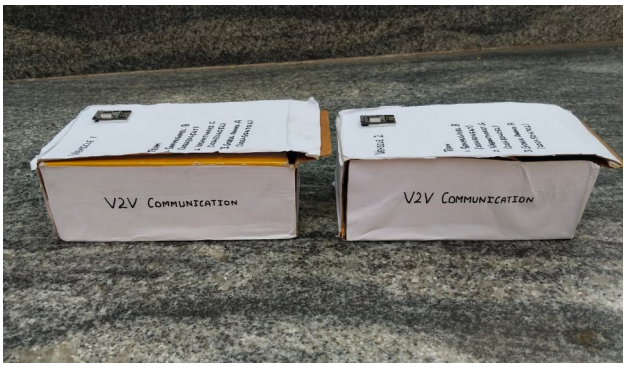


Fig. 5 Both vehicle units, closed



Fig. 6 Vehicle 1 and Vehicle 2 units with team details

9. RESULT AND CONCLUSION

A two-way V2V link was built between an Arduino Mega and an Arduino Nano. Each 1-second cycle carries the sender's ID, counter, speed and live 3-axis motion data, and the reply confirms that the link works in both directions. Using SPI for the radio, I2C for the sensor and UART for monitoring shows how the serial peripherals of a microcontroller (PIC 16F877 / dsPIC30F, Units I and V) combine to make a real embedded communication system.

Future scope: use a real wheel-speed sensor, add a sudden-braking / collision warning (buzzer + LED) from the accelerometer data, include GPS position, and port the firmware to a PIC16F877A with its MSSP (SPI/I2C) and USART modules.

APPENDIX – SOURCE CODE

A1. Vehicle 1 – Arduino Mega 2560 (mega.ino)

```
#include <SPI.h>
#include <RF24.h>
#include <Wire.h>

// NRF24L01 pins
#define CE_PIN  9
#define CSN_PIN 10

RF24 radio(CE_PIN, CSN_PIN);

// Communication address
const byte address[6] = "CAR01";

// MPU6500 I2C settings
#define MPU_ADDR       0x68
#define WHO_AM_I_REG   0x75
#define PWR_MGMT_1     0x6B
#define ACCEL_CONFIG   0x1C
#define GYRO_CONFIG    0x1B
#define ACCEL_XOUT_H   0x3B

// Both boards MUST use this exact structure
struct DataPacket {
  uint8_t vehicleID;
  uint16_t counter;
  float speed;

  uint8_t mpuPresent;

  int16_t ax_mg;
  int16_t ay_mg;
  int16_t az_mg;

  int16_t gx_dps10;
  int16_t gy_dps10;
  int16_t gz_dps10;
};

DataPacket data;
DataPacket receivedData;

bool mpuOK = false;
bool radioOK = false;

// --------------------------------
// MPU REGISTER WRITE
// --------------------------------
bool writeMPURegister(uint8_t reg, uint8_t value) {
  Wire.beginTransmission(MPU_ADDR);
  Wire.write(reg);
  Wire.write(value);

  return Wire.endTransmission() == 0;
}

// --------------------------------
// MPU REGISTER READ
// --------------------------------
bool readMPURegisters(uint8_t reg, uint8_t *buffer,
                      uint8_t length) {
  Wire.beginTransmission(MPU_ADDR);
  Wire.write(reg);

  if (Wire.endTransmission(false) != 0) {
    return false;
  }

  uint8_t received = Wire.requestFrom(
    (uint8_t)MPU_ADDR, length, (uint8_t)true
  );

  if (received != length) {
    while (Wire.available()) {
      Wire.read();
    }
    return false;
  }

  for (uint8_t i = 0; i < length; i++) {
    buffer[i] = Wire.read();
  }

  return true;
}

// --------------------------------
// CHECK AND INITIALIZE MPU6500
// --------------------------------
bool initMPU6500() {
  uint8_t whoAmI = 0;

  if (!readMPURegisters(WHO_AM_I_REG, &whoAmI, 1)) {
    Serial.println("MPU6500 I2C communication failed!");
    return false;
  }

  Serial.print("MPU WHO_AM_I = 0x");
  Serial.println(whoAmI, HEX);

  // Require the requested MPU6500 ID
  if (whoAmI != 0x70) {
    Serial.println("MPU6500 not identified: expected
0x70");
    return false;
  }

  // Wake up the sensor and select the internal clock
  if (!writeMPURegister(PWR_MGMT_1, 0x01)) {
    Serial.println("MPU power configuration failed!");
    return false;
  }

  delay(100);

  // Accelerometer: +/- 2g
  if (!writeMPURegister(ACCEL_CONFIG, 0x00)) {
    return false;
  }

  // Gyroscope: +/- 250 degrees/second
  if (!writeMPURegister(GYRO_CONFIG, 0x00)) {
    return false;
  }

  Serial.println("MPU6500 detected and configured!");
  return true;
}

// --------------------------------
// READ ACCELEROMETER + GYROSCOPE
// --------------------------------
bool readMPU6500() {
  uint8_t raw[14];

  if (!readMPURegisters(ACCEL_XOUT_H, raw, 14)) {
    return false;
  }

  int16_t ax = (int16_t)((raw[0] << 8) | raw[1]);
  int16_t ay = (int16_t)((raw[2] << 8) | raw[3]);
  int16_t az = (int16_t)((raw[4] << 8) | raw[5]);

  // raw[6] and raw[7] are temperature bytes

  int16_t gx = (int16_t)((raw[8]  << 8) | raw[9]);
  int16_t gy = (int16_t)((raw[10] << 8) | raw[11]);
  int16_t gz = (int16_t)((raw[12] << 8) | raw[13]);

  // Convert accelerometer readings to milli-g
  data.ax_mg = (int32_t)ax * 1000L / 16384;
  data.ay_mg = (int32_t)ay * 1000L / 16384;
  data.az_mg = (int32_t)az * 1000L / 16384;

  // Convert gyroscope readings to 0.1 degrees/second
  data.gx_dps10 = (int32_t)gx * 10L / 131;
  data.gy_dps10 = (int32_t)gy * 10L / 131;
  data.gz_dps10 = (int32_t)gz * 10L / 131;

  return true;
}

// --------------------------------
// SETUP
// --------------------------------
void setup() {
  Serial.begin(9600);
  Wire.begin();

  Serial.println("MEGA STARTING...");

  // MPU failure must not stop radio communication
  mpuOK = initMPU6500();

  if (!mpuOK) {
    Serial.println("MPU STATUS: NOT AVAILABLE");
  }

  if (!radio.begin()) {
    Serial.println("NRF24L01 not detected!");
    while (1) {}
```

```
  }

  radio.setPALevel(RF24_PA_LOW);
  radio.setDataRate(RF24_250KBPS);
  radio.setChannel(100);
  radio.setRetries(5, 15);

  radio.openWritingPipe(address);
  radio.openReadingPipe(1, address);

  radio.stopListening();

  data.vehicleID = 1;
  data.counter = 0;
  data.speed = 25.5;

  data.mpuPresent = 0;

  Serial.println("MEGA TWO-WAY COMMUNICATION READY");
}

// --------------------------------
// MAIN LOOP
// --------------------------------
void loop() {
  data.counter++;

  // Preserve your original example speed
  data.speed = 25.5 + (data.counter % 10);

  // Update MPU data if the sensor is available
  if (mpuOK) {
    if (!readMPU6500()) {
      mpuOK = false;
      Serial.println("MPU READ FAILED; marking
unavailable");
    }
  }

  data.mpuPresent = mpuOK ? 1 : 0;

  if (!mpuOK) {
    data.ax_mg = 0;
    data.ay_mg = 0;
    data.az_mg = 0;

    data.gx_dps10 = 0;
    data.gy_dps10 = 0;
    data.gz_dps10 = 0;
  }

  // Send Mega data to Nano
  radio.stopListening();

  bool success = radio.write(&data, sizeof(data));

  if (success) {
    Serial.print("MEGA SENT | ID: ");
    Serial.print(data.vehicleID);

    Serial.print(" | Counter: ");
    Serial.print(data.counter);

    Serial.print(" | Speed: ");
    Serial.print(data.speed);

    Serial.print(" | MPU: ");
    Serial.println(data.mpuPresent ? "OK" : "NOT
AVAILABLE");
  } else {
    Serial.println("MEGA TRANSMISSION FAILED!");
  }

  // Wait for Nano's response
  radio.startListening();

  unsigned long startTime = millis();
  bool responseReceived = false;

  while (millis() - startTime < 250) {
    if (radio.available()) {
      radio.read(&receivedData, sizeof(receivedData));
      responseReceived = true;
      break;
    }
  }

  if (responseReceived) {
    Serial.println("--- NANO DATA RECEIVED ---");

    Serial.print("Vehicle ID: ");
    Serial.println(receivedData.vehicleID);

    Serial.print("Counter: ");
    Serial.println(receivedData.counter);

    Serial.print("Speed: ");
    Serial.println(receivedData.speed);

    Serial.print("Nano MPU: ");
    Serial.println(
      receivedData.mpuPresent ? "OK" : "NOT AVAILABLE"
    );

    if (receivedData.mpuPresent) {
      Serial.print("Accel mg: ");
      Serial.print(receivedData.ax_mg);
      Serial.print(", ");
      Serial.print(receivedData.ay_mg);
      Serial.print(", ");
      Serial.println(receivedData.az_mg);

      Serial.print("Gyro 0.1 dps: ");
      Serial.print(receivedData.gx_dps10);
      Serial.print(", ");
      Serial.print(receivedData.gy_dps10);
      Serial.print(", ");
      Serial.println(receivedData.gz_dps10);
    }

    Serial.println("--------------------------");
  } else {
    Serial.println("No response received from Nano!");
  }

  delay(1000);
}
```

A2. Vehicle 2 – Arduino Nano (nano.ino)

```
#include <SPI.h>
#include <RF24.h>
#include <Wire.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>

// NRF24L01 pins
#define CE_PIN  9
#define CSN_PIN 10

RF24 radio(CE_PIN, CSN_PIN);
Adafruit_MPU6050 mpu;

// Must match Mega address
const byte address[6] = "CAR01";

// MUST MATCH THE MEGA STRUCTURE EXACTLY
struct DataPacket {
  uint8_t vehicleID;
  uint16_t counter;
  float speed;

  uint8_t mpuPresent;

  int16_t ax_mg;
  int16_t ay_mg;
  int16_t az_mg;

  int16_t gx_dps10;
  int16_t gy_dps10;
  int16_t gz_dps10;
};

DataPacket receivedData;
DataPacket responseData;

bool mpuOK = false;
uint16_t nanoCounter = 0;

// --------------------------------
// READ MPU USING ADAFRUIT LIBRARY
// --------------------------------
bool readNanoMPU() {
  sensors_event_t a, g, temp;

  if (!mpuOK) {
    return false;
  }

  mpu.getEvent(&a, &g, &temp);

  // Convert acceleration from m/s^2 to milli-g
  responseData.ax_mg = a.acceleration.x * 1000.0 /
9.80665;
```

```
  responseData.ay_mg = a.acceleration.y * 1000.0 /
9.80665;
  responseData.az_mg = a.acceleration.z * 1000.0 /
9.80665;

  // Convert angular velocity from rad/s to 0.1 degrees/s
  responseData.gx_dps10 = g.gyro.x * 1800.0 / PI;
  responseData.gy_dps10 = g.gyro.y * 1800.0 / PI;
  responseData.gz_dps10 = g.gyro.z * 1800.0 / PI;

  return true;
}

// --------------------------------
// SETUP
// --------------------------------
void setup() {
  Serial.begin(9600);
  Wire.begin();

  Serial.println("NANO STARTING...");

  // MPU failure must not stop radio communication
  mpuOK = mpu.begin(0x68);

  if (mpuOK) {
    mpu.setAccelerometerRange(MPU6050_RANGE_2_G);
    mpu.setGyroRange(MPU6050_RANGE_250_DEG);
    mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);

    Serial.println("MPU INITIALIZATION SUCCESS");
  } else {
    Serial.println("MPU NOT AVAILABLE");
    Serial.println("Continuing without MPU data");
  }

  if (!radio.begin()) {
    Serial.println("NRF24L01 not detected!");
    while (1) {}
  }

  radio.setPALevel(RF24_PA_LOW);
  radio.setDataRate(RF24_250KBPS);
  radio.setChannel(100);
  radio.setRetries(5, 15);

  radio.openWritingPipe(address);
  radio.openReadingPipe(1, address);

  radio.startListening();

  Serial.println("NANO TWO-WAY COMMUNICATION READY");
  Serial.println("Waiting for Mega data...");
}

// --------------------------------
// MAIN LOOP
// --------------------------------
void loop() {
  if (radio.available()) {

    // Receive Mega packet
    radio.read(&receivedData, sizeof(receivedData));

    Serial.println("\n--- MEGA DATA RECEIVED ---");

    Serial.print("Vehicle ID: ");
    Serial.println(receivedData.vehicleID);

    Serial.print("Counter: ");
    Serial.println(receivedData.counter);

    Serial.print("Speed: ");
    Serial.println(receivedData.speed);

    Serial.print("Mega MPU: ");
    Serial.println(
      receivedData.mpuPresent ? "OK" : "NOT AVAILABLE"
    );

    if (receivedData.mpuPresent) {
      Serial.print("Mega Accel mg: ");
      Serial.print(receivedData.ax_mg);
      Serial.print(", ");
      Serial.print(receivedData.ay_mg);
      Serial.print(", ");
      Serial.println(receivedData.az_mg);

      Serial.print("Mega Gyro 0.1 dps: ");
      Serial.print(receivedData.gx_dps10);
      Serial.print(", ");
      Serial.print(receivedData.gy_dps10);
      Serial.print(", ");
      Serial.println(receivedData.gz_dps10);
    }

    // Prepare Nano's own response
    responseData.vehicleID = 2;
    responseData.counter = ++nanoCounter;

    // Preserve the original example-speed concept
    responseData.speed = 25.5 + (nanoCounter % 10);

    responseData.mpuPresent = mpuOK ? 1 : 0;

    if (mpuOK && !readNanoMPU()) {
      mpuOK = false;
      responseData.mpuPresent = 0;
      Serial.println("MPU read failed; marking
unavailable");
    }

    if (!responseData.mpuPresent) {
      responseData.ax_mg = 0;
      responseData.ay_mg = 0;
      responseData.az_mg = 0;

      responseData.gx_dps10 = 0;
      responseData.gy_dps10 = 0;
      responseData.gz_dps10 = 0;
    }

    // Send Nano data back to Mega
    radio.stopListening();

    bool success = radio.write(
      &responseData, sizeof(responseData)
    );

    if (success) {
      Serial.println("\n--- NANO DATA SENT ---");

      Serial.print("Vehicle ID: ");
      Serial.println(responseData.vehicleID);

      Serial.print("Counter: ");
      Serial.println(responseData.counter);

      Serial.print("Speed: ");
      Serial.println(responseData.speed);

      Serial.print("Nano MPU: ");
      Serial.println(
        responseData.mpuPresent ? "OK" : "NOT AVAILABLE"
      );

      if (responseData.mpuPresent) {
        Serial.print("Accel mg: ");
        Serial.print(responseData.ax_mg);
        Serial.print(", ");
        Serial.print(responseData.ay_mg);
        Serial.print(", ");
        Serial.println(responseData.az_mg);

        Serial.print("Gyro 0.1 dps: ");
        Serial.print(responseData.gx_dps10);
        Serial.print(", ");
        Serial.print(responseData.gy_dps10);
        Serial.print(", ");
        Serial.println(responseData.gz_dps10);
      }

      Serial.println("-----------------------");

    } else {
      Serial.println("Nano transmission failed!");
    }

    // Return to receive mode
    radio.startListening();
  }
}
```

A3. V2V Web-Serial dashboard (V2V_Dashboard.html)

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>V2V Blind-Curve Alert</title>
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?family=Barlow:wght@400;500;600;700&family=Barlow+Semi+Condensed:wght@500;600;700&family=IBM+Plex+Mono:wght@400;500&display=swap" rel="stylesheet">
<style>
  :root{
    --bg:#E8EBEF; --panel:#FFFFFF; --soft:#F4F6F8; --line:#DFE3E8;
    --text:#16191D; --muted:#646B75; --dim:#9AA1AB;
    --v1:#2563C9; --v2:#E0701B;
    --ok:#15803D; --okbg:#E7F5EC; --amber:#B86E00; --amberbg:#FFF4DC; --red:#D92D20; --redbg:#FDECEA;
    --sans:"Barlow","Segoe UI",Roboto,Arial,sans-serif;
    --cond:"Barlow Semi Condensed","Barlow","Arial Narrow",sans-serif;
    --mono:"IBM Plex Mono",Consolas,"Courier New",monospace;
    --shadow:0 1px 2px rgba(16,24,40,.05),0 2px 8px rgba(16,24,40,.06);
  }
  *{box-sizing:border-box}
  html,body{margin:0;background:var(--bg);color:var(--text)}
  body{font:15px/1.45 var(--sans);-webkit-font-smoothing:antialiased}
  .wrap{max-width:1380px;margin:0 auto;padding:0 20px 24px}
  button{font:inherit}

  /* ---------- top bar ---------- */
  header.top{display:grid;grid-template-columns:1fr auto 1fr;align-items:center;gap:14px;padding:14px 0}
  .brand{display:flex;gap:12px;align-items:center;min-width:0}
  .brand svg{flex:none}
  .brand h1{margin:0;font:700 19px/1.1 var(--cond);letter-spacing:.01em}
  .brand p{margin:2px 0 0;color:var(--muted);font-size:13px}
  .clock{font:600 20px var(--cond);font-variant-numeric:tabular-nums;color:var(--text);letter-spacing:.02em}
  .controls{display:flex;gap:8px;align-items:center;flex-wrap:wrap;justify-content:flex-end}
  .btn{border:1px solid var(--line);background:var(--panel);color:var(--text);padding:9px 14px;border-radius:8px;cursor:pointer;font-weight:600;font-size:14px;line-height:1;box-shadow:var(--shadow)}
  .btn:hover{border-color:#c4cad2}
  .btn.primary{background:var(--text);color:#fff;border-color:var(--text)}
  .btn.primary:hover{background:#000}
  .btn.on{color:var(--v1);border-color:#b9cbec}
  .chips{display:flex;gap:6px;flex-wrap:wrap}
  .chip{display:inline-flex;align-items:center;gap:8px;background:var(--panel);border:1px solid var(--line);border-radius:8px;padding:5px 5px 5px 10px;font-size:13px;font-weight:600}
  .chip i{width:8px;height:8px;border-radius:50%;display:block}
  .chip button{border:0;background:var(--soft);color:var(--muted);border-radius:6px;padding:4px 8px;cursor:pointer;font-size:12px}
  .chip button:hover{color:var(--text)}

  /* ---------- scene ---------- */
  .scene{position:relative;border-radius:16px;overflow:hidden;background:#EEF1EA;height:430px;box-shadow:var(--shadow);border:1px solid var(--line)}
  .scene canvas{display:block;width:100%;height:100%}
  .hud{position:absolute;left:14px;top:14px;display:flex;gap:8px;flex-wrap:wrap;pointer-events:none}
  .tag{display:inline-flex;align-items:center;gap:8px;background:rgba(255,255,255,.92);border:1px solid var(--line);border-radius:8px;padding:7px 11px;font-size:13px;color:var(--muted);box-shadow:var(--shadow)}
  .tag b{color:var(--text);font-weight:700;letter-spacing:.02em}
  .tag i{width:8px;height:8px;border-radius:50%;background:var(--dim);display:block}
  .tag.bad b{color:var(--red)} .tag.bad i{background:var(--red)}
  .tag.good b{color:var(--ok)} .tag.good i{background:var(--ok)}
  .tag.warn b{color:var(--amber)} .tag.warn i{background:var(--amber)}
  .scene .title{position:absolute;right:16px;top:16px;text-align:right;pointer-events:none}
  .scene .title b{display:block;font:700 15px var(--cond);color:#3d4a37;letter-spacing:.02em}
  .scene .title span{font-size:12px;color:#6c7a64}
  .scene .note{position:absolute;left:16px;bottom:10px;font-size:11.5px;color:#6c7a64;pointer-events:none}
  .idle{position:absolute;inset:0;display:flex;align-items:center;justify-content:center;background:rgba(238,241,234,.55);padding:16px}
  .idle .box{background:var(--panel);border-radius:14px;box-shadow:0 8px 30px rgba(16,24,40,.12);padding:22px 24px;text-align:center;max-width:430px}
  .idle strong{display:block;font:700 20px var(--cond)}
  .idle span{display:block;color:var(--muted);font-size:14px;margin:6px 0 14px}
  .idle .row{display:flex;gap:8px;justify-content:center;flex-wrap:wrap}

  /* ---------- cluster ---------- */
  .cluster{display:grid;grid-template-columns:1fr 1.08fr 1fr;gap:14px;margin-top:14px}
  .card{background:var(--panel);border:1px solid var(--line);border-radius:16px;padding:16px 18px;box-shadow:var(--shadow);min-width:0;transition:border-color .2s,background .2s}
  .card h2{margin:0;font:600 12px var(--sans);letter-spacing:.09em;text-transform:uppercase;color:var(--muted)}
  .vh{display:flex;justify-content:space-between;align-items:center;gap:10px}
  .vname{display:flex;gap:10px;align-items:center}
  .swatch{width:5px;height:30px;border-radius:3px;background:var(--c)}
  .vname b{display:block;font:700 18px/1 var(--cond)}
  .vname small{display:block;color:var(--muted);font-size:13px;margin-top:3px}
  .pill{font:700 12px/1 var(--cond);letter-spacing:.07em;padding:6px 10px;border-radius:6px;text-transform:uppercase;background:var(--soft);color:var(--muted);white-space:nowrap}
  .pill.ok{background:var(--okbg);color:var(--ok)}
  .pill.warn{background:var(--amberbg);color:var(--amber)}
  .pill.brake{background:var(--red);color:#fff;animation:blink .55s steps(2) infinite}
  @keyframes blink{50%{opacity:.35}}
  .gauge{display:block;width:100%;max-width:330px;margin:4px auto 0}
  .gauge .num{font:600 50px var(--cond);font-variant-numeric:tabular-nums}
  .card.braking{border-color:var(--red);box-shadow:0 0 0 1px var(--red) inset,var(--shadow)}
  .stats{display:grid;grid-template-columns:1fr 1fr;gap:8px;margin-top:6px}
  .stat{background:var(--soft);border-radius:10px;padding:9px 12px;min-width:0}
  .stat span{display:block;font-size:12px;color:var(--muted)}
  .stat b{display:block;font:600 18px/1.2 var(--cond);font-variant-numeric:tabular-nums;white-space:nowrap;overflow:hidden;text-overflow:ellipsis}

  .center{display:flex;flex-direction:column;text-align:center}
  .center .state{flex:1;display:flex;flex-direction:column;align-items:center;justify-content:center;padding:18px 6px 14px;gap:6px}
  .center .icon{width:96px;height:96px}
  .center .t{font:700 26px/1.1 var(--cond);margin-top:6px}
  .center .s{color:var(--muted);font-size:14.5px;max-width:360px}
  .center.alarm{background:var(--redbg);border-color:var(--red);box-shadow:0 0 0 1px var(--red) inset,var(--shadow)}
  .center.alarm .t{color:var(--red)}
  .center.alarm .icon{animation:throb 1s ease-in-out infinite}
  .center.lost{background:#FFFBF2;border-color:#f0d49a}
  .center.lost .t{color:var(--amber)}
  @keyframes throb{50%{transform:scale(1.07)}}
  .linkrow{display:grid;grid-template-columns:repeat(3,1fr);border-top:1px solid var(--line);padding-top:12px;gap:6px}
  .linkrow span{display:block;font-size:12px;color:var(--muted)}
  .linkrow b{display:block;font:600 22px/1.15 var(--cond);font-variant-numeric:tabular-nums}
  .center.alarm .linkrow{border-color:#f3b8b2}
  .spec{font-size:11.5px;color:var(--dim);margin-top:8px}

  /* ---------- lower ---------- */
  .lower{display:grid;grid-template-columns:2fr 1fr;gap:14px;margin-top:14px}
  .ch-head{display:flex;justify-content:space-between;align-items:center;gap:12px;flex-wrap:wrap}
  .legend{display:flex;gap:14px;font-size:13px;color:var(--muted);flex-wrap:wrap}
  .legend span{display:inline-flex;align-items:center;gap:6px}
  .legend i{width:14px;height:3px;border-radius:2px;display:block;background:var(--c)}
  .legend i.dash{background:repeating-linear-gradient(90deg,var(--red) 0 4px,transparent 4px 7px)}

  #chart{display:block;width:100%;height:240px;margin-top:10px}
  .events{list-style:none;margin:8px 0 0;padding:0;max-height:250px;overflow:auto}
  .events li{display:grid;grid-template-columns:auto 8px 1fr;gap:10px;align-items:baseline;padding:8px 0;border-top:1px solid var(--line);font-size:14px}
  .events li:first-child{border-top:0}
  .events time{font:500 12px var(--mono);color:var(--dim)}
  .events i{width:8px;height:8px;border-radius:50%;background:var(--dim);display:block;transform:translateY(-1px)}
  .events .brake i{background:var(--red)} .events .brake{color:var(--red);font-weight:600}
  .events .ok i{background:var(--ok)} .events .err i{background:var(--amber)} .events .err{color:var(--amber)}
  .events .x{color:var(--dim);font-size:12px;margin-left:4px}
  .events .empty{display:block;color:var(--dim)}

  details.console{margin-top:14px;background:var(--panel);border:1px solid var(--line);border-radius:16px;box-shadow:var(--shadow)}
  details.console summary{cursor:pointer;padding:14px 18px;font:600 12px var(--sans);letter-spacing:.09em;text-transform:uppercase;color:var(--muted);list-style:none;display:flex;justify-content:space-between}
  details.console summary::-webkit-details-marker{display:none}
  details.console summary::after{content:"Show";letter-spacing:.02em;text-transform:none;font-weight:500}
  details.console[open] summary::after{content:"Hide"}
  details.console pre{margin:0;padding:12px 18px 16px;border-top:1px solid var(--line);max-height:260px;overflow:auto;font:12.5px/1.55 var(--mono);color:#3a4048;white-space:pre-wrap;word-break:break-word}
  .t1{color:var(--v1)} .t2{color:var(--v2)} .tx{color:var(--dim)}
  footer{margin-top:16px;color:var(--dim);font-size:12px;display:flex;justify-content:space-between;gap:10px;flex-wrap:wrap}

  @media (max-width:1100px){
    .cluster{grid-template-columns:1fr 1fr}
    .center{grid-column:span 2;order:-1}
    .lower{grid-template-columns:1fr}
  }
  @media (max-width:720px){
    .wrap{padding:0 16px 20px}
    header.top{grid-template-columns:1fr auto}
    .clock{display:none}
    .controls{grid-column:span 2;justify-content:flex-start}
    .scene{height:330px}
    .scene .title,.scene .note{display:none}
    .cluster{grid-template-columns:minmax(0,1fr)}
    .center{grid-column:auto}
    .center .t{font-size:22px}
    #chart{height:200px}
  }
</style>
</head>
<body>
<div class="wrap">

  <header class="top">
    <div class="brand">
      <svg width="36" height="36" viewBox="0 0 36 36" aria-hidden="true">
        <path d="M18 3 L33 30 H3 Z" fill="#fff" stroke="#D92D20" stroke-width="3.2" stroke-linejoin="round"/>
        <path d="M12 26 C12 18, 22 20, 21 12" fill="none" stroke="#16191D" stroke-width="2.6" stroke-linecap="round"/>
      </svg>
      <div>
        <h1>V2V Blind-Curve Alert</h1>
        <p>Opposite-direction vehicles · mountain roads</p>
      </div>
    </div>
    <div class="clock" id="clock">--:--</div>
    <div class="controls">
      <div class="chips" id="chips"></div>
      <button class="btn primary" id="btnConnect">Connect board</button>
      <button class="btn" id="btnDemo">Demo</button>
      <button class="btn on" id="btnSound" title="Beep on emergency braking">Sound on</button>
    </div>
  </header>
```

```
  <section class="scene">
    <canvas id="road"></canvas>
    <div class="hud">
      <span class="tag" id="tagLos"><i></i>Line of sight <b>—</b></span>
      <span class="tag" id="tagRadio"><i></i>V2V radio <b>Offline</b></span>
    </div>
    <div class="title"><b>Blind hairpin bend</b><span>Vehicle 1    uphill · Vehicle 2   downhill</span></div>
    <div class="note">Car positions are illustrative · braking, warnings and radio link are live from the boards</div>
    <div class="idle" id="idle">
      <div class="box">
        <strong id="idleTitle">No board connected</strong>
        <span id="idleMsg">Plug the Mega or Nano into this computer, close the Arduino Serial Monitor, then press Connect board.</span>
        <div class="row">
          <button class="btn primary" data-act="connect">Connect board</button>
          <button class="btn" data-act="demo">Run demo</button>
        </div>
      </div>
    </div>
  </section>

  <section class="cluster">
    <article class="card" id="card1" style="--c:var(--v1)"></article>

    <article class="card center" id="centre">
      <h2>Hazard warning</h2>
      <div class="state">
        <svg class="icon" id="cIcon" viewBox="0 0 96 96" aria-hidden="true"></svg>
        <div class="t" id="cTitle">V2V standby</div>
        <div class="s" id="cSub">Connect a vehicle board to start.</div>
      </div>
      <div class="linkrow">
        <div><span>TX acknowledged</span><b id="txPct">—</b></div>
        <div><span>Packets / s</span><b id="rxRate">0.0</b></div>
        <div><span>Last packet</span><b id="rxAge">—</b></div>
      </div>
      <div class="spec">NRF24L01 · channel 100 · 250 kbps · <span id="rxCount">0</span> packets received · <span id="txCount">0 / 0</span> TX ok / failed</div>
    </article>

    <article class="card" id="card2" style="--c:var(--v2)"></article>
  </section>

  <section class="lower">
    <article class="card">
      <div class="ch-head">
        <h2>Acceleration along X · last 20 s</h2>
        <div class="legend">
          <span style="--c:var(--v1)"><i></i>Vehicle 1</span>
          <span style="--c:var(--v2)"><i></i>Vehicle 2</span>
          <span><i class="dash"></i>Brake threshold -4.0</span>
        </div>
      </div>
      <canvas id="chart"></canvas>
    </article>
    <article class="card">
      <h2>Events</h2>
      <ul class="events" id="events"><li class="empty">Nothing yet.</li></ul>
    </article>
  </section>

  <details class="console" id="console">
    <summary>Serial monitor</summary>
    <pre id="raw"></pre>
  </details>

  <footer>
    <span>Reads the boards' Serial output at 9600 baud · Chrome or Edge on a computer</span>
    <span>Webora</span>
  </footer>
</div>

<script>
(() => {
  "use strict";

  const BAUD = 9600;               // Serial.begin(9600) in both sketches
  const TH = -4.0;                 // m/s² along X (≈ -408 mg) counts as hard braking
  const LIVE_MS = 2500;            // data older than this counts as lost
  const HOLD_MS = 1200;            // keep the brake alert visible a moment after it clears
  const MODEL = {1: "Arduino Mega", 2: "Arduino Nano"};
  const COLOR = {1: "#2563C9", 2: "#E0701B"};
  const $ = id => document.getElementById(id);
  const now = () => performance.now();

  // ---------------- state ----------------
  const V = {};
  function freshVehicle(id) {
    return {id, ax: null, gx: null, speed: null, counter: null, mpu: null, em: false, emUntil: 0, last: 0, via: "",
            hist: [], peak: null, brakes: 0, tx: [], txOk: 0, txFail: 0,
            lastWave: 0, lastBeep: 0, gauge: 0};
  }
  const link = {rx: 0, rxTimes: [], lastRx: 0, lostLogged: false};
  const ports = [];
  const events = [];
  const raw = [];
  let sound = true, demo = null, rawDirty = false, eventsDirty = false;

  function resetAll() {
    V[1] = freshVehicle(1); V[2] = freshVehicle(2);
    Object.assign(link, {rx: 0, rxTimes: [], lastRx: 0, lostLogged: false});
    events.length = 0; raw.length = 0; eventsDirty = rawDirty = true;
  }
  resetAll();

  // ---------------- data updates ----------------
  function feed(id, ax, via, gx) {
    const v = V[id]; if (!v || !isFinite(ax)) return;
    const t = now();
    v.ax = ax; v.last = t; v.via = via;
    if (gx !== undefined && isFinite(gx)) v.gx = gx;
    v.hist.push({t, ax});
    while (v.hist.length && t - v.hist[0].t > 22000) v.hist.shift();
    if (v.peak === null || ax < v.peak) v.peak = ax;
    setEm(id, ax < TH);
  }

  function setEm(id, on) {
    const v = V[id]; if (!v) return;
    const t = now();
    if (on) {
      if (!v.em && t > v.emUntil) {
        v.brakes++;
        log("brake", `Vehicle ${id} emergency braking (${fmt(v.ax)} m/s²)`);
        if (t - v.lastBeep > 1500) { beep(); v.lastBeep = t; }
      }
      v.emUntil = t + HOLD_MS;
    }
    v.em = on;
  }

  function tx(id, ok) {
    const v = V[id]; if (!v) return;
    v.tx.push(ok); if (v.tx.length > 50) v.tx.shift();
    ok ? v.txOk++ : v.txFail++;
  }

  function rx(fromId) {
    const t = now();
    link.rx++; link.lastRx = t; link.rxTimes.push(t);
    while (link.rxTimes.length && t - link.rxTimes[0] > 5000) link.rxTimes.shift();
    if (link.lostLogged) { log("ok", "Radio link restored"); link.lostLogged = false; }
    const v = V[fromId];
    if (v && t - v.lastWave > 320) { v.lastWave = t; waves.push({from: fromId, t0: t, em: v.em}); }
  }

  // ---------------- parser (matches the Mega + Nano two-way sketches' Serial.print text) ----------------
  const G = 9.80665;
  function commit(P) {
    const b = P.block; P.block = null;
    if (!b || !V[b.id]) return;
    const v = V[b.id];
    if (b.speed !== undefined) v.speed = b.speed;
    if (b.counter !== undefined) v.counter = b.counter;
    if (b.mpu !== undefined) v.mpu = b.mpu;
    if (b.mpu && b.ax !== undefined) feed(b.id, b.ax, b.via, b.gx);
    else {
      v.last = now(); v.via = b.via;
      if (b.mpu === false) { v.ax = null; v.gx = null; setEm(b.id, false); }
    }
    if (b.via === "Radio") rx(b.id);
  }

  function parse(P, input) {
    const line = input.replace(/\r/g, "").trim();
    if (!line) return;
    pushRaw(P, line);
    let m;

    // packet blocks: "--- MEGA DATA RECEIVED ---" (on Nano), "--- NANO DATA SENT ---" (on Nano), "--- NANO DATA RECEIVED ---" (on Mega)
    if ((m = line.match(/^--- (MEGA|NANO) DATA (RECEIVED|SENT) ---$/))) {
      commit(P);
      const from = m[1] === "MEGA" ? 1 : 2, sent = m[2] === "SENT";
      identify(P, sent ? from : 3 - from);
      if (sent) tx(from, true);
      P.block = {id: from, via: sent ? "USB" : "Radio"};
      return;
    }
    if (P.block) {
      const b = P.block;
      if ((m = line.match(/^Vehicle ID: (\d+)/))) { b.id = +m[1]; return; }
      if ((m = line.match(/^Counter: (\d+)/))) { b.counter = +m[1]; return; }
      if ((m = line.match(/^Speed: (-?\d+(?:\.\d+)?)/))) { b.speed = +m[1]; return; }
      if ((m = line.match(/MPU: (OK|NOT AVAILABLE)$/))) { b.mpu = m[1] === "OK"; if (!b.mpu) commit(P); return; }
      if ((m = line.match(/Accel mg: (-?\d+), (-?\d+), (-?\d+)/))) { b.ax = +m[1] * G / 1000; return; }
      if ((m = line.match(/Gyro 0\.1 dps: (-?\d+), (-?\d+), (-?\d+)/))) { b.gx = +m[1] / 10; commit(P); return; }
      commit(P);
    }

    // Mega one-line send report (the Mega does not print its own accel values)
    if ((m = line.match(/^MEGA SENT \| ID: (\d+) \| Counter: (\d+) \| Speed: (-?\d+(?:\.\d+)?) \| MPU: (OK|NOT AVAILABLE)/))) {
      const id = +m[1], v = V[id]; identify(P, id); tx(id, true);
      if (v) { v.counter = +m[2]; v.speed = +m[3]; v.mpu = m[4] === "OK"; v.last = now(); if (!v.via) v.via = "USB"; }
      return;
    }
    if (/^MEGA TRANSMISSION FAILED/.test(line)) { identify(P, 1); tx(1, false); return; }
    if (/^Nano transmission failed/.test(line)) { identify(P, 2); tx(2, false); return; }
    if (/^No response received from Nano/.test(line)) { identify(P, 1); log("err", "Mega: no reply from the Nano"); return; }

    // boot / status messages
    if (/^MEGA STARTING/.test(line)) { identify(P, 1); log("info", "Vehicle 1 (Mega) booting"); return; }
    if (/^NANO STARTING/.test(line)) { identify(P, 2); log("info", "Vehicle 2 (Nano) booting"); return; }
    if (/TWO-WAY COMMUNICATION READY/.test(line)) { log("ok", `${P.label}: radio ready`); return; }
    if (/NOT AVAILABLE|not detected|not identified|communication failed|configuration failed|READ FAILED|read failed/i.test(line)) { log("err", `${P.label}: ${line}`); return; }
    if (/MPU INITIALIZATION SUCCESS|MPU6500 detected|WHO_AM_I/.test(line)) { log("info", `${P.label}: ${line}`); return; }
  }

  function identify(P, id) {
    if (!V[id] || P.boardId === id) return;
    P.boardId = id;
    P.label = `Vehicle ${id}`;
    renderChips();
  }

  // ---------------- Web Serial ----------------
  let portSeq = 0;
  function newPort(port, isDemo) {
    return {key: ++portSeq, port, demo: !!isDemo, open: true, reader: null,
            boardId: 0, label: `Port ${portSeq}`, inRemote: false, remote: {}, pending: null};
  }

  async function connect() {
    if (!("serial" in navigator)) {
      setIdle("Open in Chrome or Edge", "This browser can't read USB serial. Use Google Chrome or Microsoft Edge on a laptop or desktop.");
      log("err", "Web Serial not available in this browser — use Chrome or Edge");
      return;
    }
    let port;
    try { port = await navigator.serial.requestPort(); } catch { return; }   // user closed the picker
    if (ports.some(p => p.port === port)) { log("info", "That port is already connected"); return; }
    openPort(port, false);
  }

  // open a port; quiet = automatic attempt (remembered board on page load / re-plug)
  async function openPort(port, quiet) {
    if (ports.some(p => p.port === port)) return;
    if (demo) stopDemo();
    try { await port.open({baudRate: BAUD}); }
    catch (e) {
```

```
      log("err", "Could not open the port — close Arduino Serial Monitor / Plotter and try again");
      return;
    }
    const P = newPort(port);
    ports.push(P);
    log("ok", "Board connected over USB");
    renderChips();
    readLoop(P);
  }

  async function readLoop(P) {
    const dec = new TextDecoder();
    let buf = "";
    while (P.open && P.port.readable) {
      P.reader = P.port.readable.getReader();
      try {
        for (;;) {
          const {value, done} = await P.reader.read();
          if (done) break;
          buf += dec.decode(value, {stream: true});
          let i;
          while ((i = buf.indexOf("\n")) >= 0) { parse(P, buf.slice(0, i)); buf = buf.slice(i + 1); }
          if (buf.length > 4096) buf = "";
        }
      } catch (e) {
        // framing / buffer errors are recoverable; a lost device ends the loop below
      } finally {
        try { P.reader.releaseLock(); } catch {}
      }
      if (!P.open) break;
    }
    if (P.open) disconnect(P, true);
  }

  async function disconnect(P, lost) {
    if (!P.open) return;
    P.open = false;
    if (P.demo) { stopDemo(); return; }
    try { await P.reader?.cancel(); } catch {}
    try { await P.port.close(); } catch {}
    ports.splice(ports.indexOf(P), 1);
    log(lost ? "err" : "info", `${P.label} ${lost ? "unplugged" : "disconnected"}`);
    renderChips();
  }

  if ("serial" in navigator) {
    navigator.serial.addEventListener("disconnect", e => {
      const P = ports.find(p => p.port === e.target);
      if (P) disconnect(P, true);
    });
    // boards allowed once before are opened automatically — on page load and when plugged back in
    navigator.serial.addEventListener("connect", e => openPort(e.target, true));
    navigator.serial.getPorts().then(list => list.forEach(port => openPort(port, true))).catch(() => {});
  }

  // ---------------- demo (feeds the parser with the sketches' exact output) ----------------
  function startDemo() {
    if (demo) return stopDemo();
    resetAll();
    const A = newPort(null, true), B = newPort(null, true);
    A.label = "Demo · Mega"; B.label = "Demo · Nano";
    ports.push(A, B);
    const sim = {1: {ax: 0, brakeT: 0}, 2: {ax: 0, brakeT: 0}};
    let tick = 0, next = 1;
    const boot = [
      [A, "MEGA STARTING..."], [A, "MPU WHO_AM_I = 0x70"], [A, "MPU6500 detected and configured!"], [A, "MEGA TWO-WAY COMMUNICATION READY"],
      [B, "NANO STARTING..."], [B, "MPU INITIALIZATION SUCCESS"], [B, "NANO TWO-WAY COMMUNICATION READY"], [B, "Waiting for Mega data..."]
    ];
    boot.forEach(([P, l]) => parse(P, l));
    const gauss = () => (Math.random() + Math.random() + Math.random() - 1.5) * 0.35;
    const mg = ax => Math.round(ax * 1000 / 9.80665);
    const block = (id, n, ax) => [`Vehicle ID: ${id}`, `Counter: ${n}`, `Speed: ${(25.5 + n % 10).toFixed(2)}`,
      `${id === 1 ? "Mega" : "Nano"} MPU: OK`, `${id === 1 ? "Mega " : ""}Accel mg: ${mg(ax)}, ${Math.round(gauss() * 40)}, ${1000 + Math.round(gauss() * 20)}`,
      `${id === 1 ? "Mega " : ""}Gyro 0.1 dps: ${Math.round(gauss() * 30)}, ${Math.round(gauss() * 30)}, ${Math.round(gauss() * 30)}`];
    let n1 = 0, n2 = 0;
    const id = setInterval(() => {
      tick++;
      if (tick % 40 === 20) { sim[next].brakeT = 6; next = next === 1 ? 2 : 1; }
      for (const k of [1, 2]) {
        const s = sim[k];
        const target = s.brakeT > 0 ? -6.2 - Math.random() * 1.2 : 0.25 + Math.sin(tick / 9 + k) * 0.6;
        s.ax += (target - s.ax) * 0.6; s.ax += gauss();
        if (s.brakeT > 0) s.brakeT--;
      }
      n1++;
      const ok1 = Math.random() > 0.03;
      parse(A, ok1 ? `MEGA SENT | ID: 1 | Counter: ${n1} | Speed: ${(25.5 + n1 % 10).toFixed(2)} | MPU: OK` : "MEGA TRANSMISSION FAILED!");
      if (!ok1) { parse(A, "No response received from Nano!"); return; }
      // Nano (Vehicle 2) serial output
      ["", "--- MEGA DATA RECEIVED ---", ...block(1, n1, sim[1].ax)].forEach(l => parse(B, l));
      n2++;
      if (Math.random() > 0.03) {
        ["", "--- NANO DATA SENT ---", ...block(2, n2, sim[2].ax), "------------------------"].forEach(l => parse(B, l));
        // Mega (Vehicle 1) receives the reply
        ["--- NANO DATA RECEIVED ---", ...block(2, n2, sim[2].ax), "--------------------------"].forEach(l => parse(A, l));
      } else { parse(B, "Nano transmission failed!"); parse(A, "No response received from Nano!"); }
    }, 400);
    demo = {id, A, B};
    log("info", "Demo running — simulated output from both boards");
    $("btnDemo").textContent = "Stop demo"; $("btnDemo").classList.add("on");
    renderChips();
  }

  function stopDemo() {
    if (!demo) return;
    clearInterval(demo.id);
    for (const P of [demo.A, demo.B]) { const i = ports.indexOf(P); if (i >= 0) ports.splice(i, 1); }
    demo = null;
    resetAll();
    $("btnDemo").textContent = "Demo"; $("btnDemo").classList.remove("on");
    renderChips();
  }

  // ---------------- sound ----------------
  let actx = null;
  function beep() {
    if (!sound) return;
    try {
      actx = actx || new (window.AudioContext || window.webkitAudioContext)();
      const t = actx.currentTime;
      for (let k = 0; k < 3; k++) {
        const o = actx.createOscillator(), g = actx.createGain(), s = t + k * 0.16;
        o.type = "square"; o.frequency.value = 1046;
        g.gain.setValueAtTime(0.0001, s);
        g.gain.exponentialRampToValueAtTime(0.09, s + 0.01);
        g.gain.exponentialRampToValueAtTime(0.0001, s + 0.11);
        o.connect(g).connect(actx.destination);
        o.start(s); o.stop(s + 0.12);
      }
    } catch {}
  }

  // ---------------- log + raw console ----------------
  function clock() { return new Date().toTimeString().slice(0, 8); }
  function log(kind, text) {
    const t = now(), top = events[0];
    if (top && top.text === text && t - top.t < 4000) { top.n++; top.t = t; top.time = clock(); }
    else { events.unshift({kind, text, t, time: clock(), n: 1}); if (events.length > 80) events.pop(); }
    eventsDirty = true;
  }
  function pushRaw(P, line) {
    raw.push([P.boardId, P.label, line]);
    if (raw.length > 300) raw.shift();
    rawDirty = true;
  }
  const esc = s => s.replace(/[&<>]/g, c => ({"&": "&amp;", "<": "&lt;", ">": "&gt;"}[c]));

  // ---------------- cluster gauges ----------------
  const fmt = n => (n < 0 ? "-" : "") + Math.abs(n).toFixed(2);
  const ago = t => { const s = (now() - t) / 1000; return s < 1 ? "now" : s < 60 ? s.toFixed(1) + " s" : Math.floor(s / 60) + " min"; };
  const GC = {x: 150, y: 132, r: 108};
  const ang = v => (-120 + (Math.max(-10, Math.min(10, v)) + 10) / 20 * 240) * Math.PI / 180;
  const pt = (a, r) => [GC.x + r * Math.sin(a), GC.y - r * Math.cos(a)];
  function arc(v0, v1, r) {
    const a0 = ang(Math.min(v0, v1)), a1 = ang(Math.max(v0, v1));
    const [x0, y0] = pt(a0, r), [x1, y1] = pt(a1, r);
    return `M${x0.toFixed(1)} ${y0.toFixed(1)} A${r} ${r} 0 ${a1 - a0 > Math.PI ? 1 : 0} 1 ${x1.toFixed(1)} ${y1.toFixed(1)}`;
  }

  function buildCard(id) {
    let ticks = "";
    for (let v = -10; v <= 10; v++) {
      const a = ang(v), major = v % 5 === 0;
      const [x0, y0] = pt(a, GC.r - (major ? 14 : 8)), [x1, y1] = pt(a, GC.r);
      ticks += `<line x1="${x0.toFixed(1)}" y1="${y0.toFixed(1)}" x2="${x1.toFixed(1)}" y2="${y1.toFixed(1)}" stroke="${v <= TH ? "#D92D20" : "#8A919C"}" stroke-width="${major ? 2 : 1.2}"/>`;
      if (major) {
        const [lx, ly] = pt(a, GC.r - 28);
        ticks += `<text x="${lx.toFixed(1)}" y="${(ly + 4).toFixed(1)}" text-anchor="middle" font-size="12" fill="#646B75" font-family="Barlow, sans-serif">${v > 0 ? "+" + v : v < 0 ? "−" + -v : 0}</text>`;
      }
    }
    $("card" + id).innerHTML = `
      <div class="vh">
        <div class="vname"><span class="swatch"></span><div><b>Vehicle ${id}</b><small>${MODEL[id]} · ${id === 1 ? "uphill" : "downhill"}</small></div></div>
        <span class="pill" data-k="pill">No data</span>
      </div>
      <svg class="gauge" viewBox="-24 0 348 236" aria-label="Vehicle ${id} acceleration">
        <path d="${arc(-10, 10, GC.r + 6)}" fill="none" stroke="#EDF0F3" stroke-width="10" stroke-linecap="round"/>
        <path d="${arc(-10, TH, GC.r + 6)}" fill="none" stroke="#F7C8C3" stroke-width="10" stroke-linecap="round"/>
        <path data-k="val" d="" fill="none" stroke="${COLOR[id]}" stroke-width="10" stroke-linecap="round"/>
        ${ticks}
        <g data-k="needle"><path d="M${GC.x} ${GC.y - GC.r - 14} l-7 -12 h14 z" fill="#16191D"/></g>
        <text data-k="num" class="num" x="${GC.x}" y="${GC.y + 18}" text-anchor="middle" fill="#16191D">—</text>
        <text x="${GC.x}" y="${GC.y + 42}" text-anchor="middle" font-size="14" fill="#646B75" font-family="Barlow, sans-serif">m/s² along X</text>
        <text data-k="g" x="${GC.x}" y="${GC.y + 64}" text-anchor="middle" font-size="13" fill="#9AA1AB" font-family="Barlow, sans-serif"></text>
        <text x="${pt(ang(-9.6), GC.r + 30)[0]}" y="${pt(ang(-9.6), GC.r + 30)[1] + 4}" text-anchor="middle" font-size="11" font-weight="700" letter-spacing="1" fill="#D92D20" font-family="Barlow, sans-serif">BRAKE</text>
      </svg>
      <div class="stats">
        <div class="stat"><span>Hardest braking</span><b data-k="peak">—</b></div>
        <div class="stat"><span>Emergency events</span><b data-k="brakes">0</b></div>
        <div class="stat"><span>TX success</span><b data-k="tx">—</b></div>
        <div class="stat"><span>Signal</span><b data-k="via">—</b></div>
      </div>`;
  }
  buildCard(1); buildCard(2);
  const q = (id, k) => $("card" + id).querySelector(`[data-k="${k}"]`);

  function isBraking(v) { return v.em || now() < v.emUntil; }
  function isLive(v) { return v.last && now() - v.last < LIVE_MS; }

  function drawGauges(dt) {
    for (const id of [1, 2]) {
      const v = V[id], live = isLive(v), br = live && isBraking(v);
      const target = v.ax === null ? 0 : v.ax;
      v.gauge += (target - v.gauge) * Math.min(1, dt * 9);
      q(id, "needle").setAttribute("transform", `rotate(${(ang(v.gauge) * 180 / Math.PI).toFixed(2)} ${GC.x} ${GC.y})`);
      const val = q(id, "val");
      val.setAttribute("d", v.ax === null || Math.abs(v.gauge) < 0.05 ? "" : arc(0, v.gauge, GC.r + 6));
      val.setAttribute("stroke", br ? "#D92D20" : COLOR[id]);
      const num = q(id, "num");
      num.textContent = v.ax === null ? "—" : fmt(v.ax);
      num.setAttribute("fill", br ? "#D92D20" : "#16191D");
```

```
      q(id, "g").textContent = (v.speed === null ? "" : `Speed ${v.speed.toFixed(1)} · `) + (v.mpu === false ? "MPU not available" : v.ax === null ? (v.speed === null ? "" : "accel via the other board") : `${fmt(v.ax / 9.80665)} g`);
    }
  }

  // ---------------- centre display icons ----------------
  const ICON = {
    standby: `<circle cx="48" cy="48" r="44" fill="#F4F6F8"/><g fill="none" stroke="#9AA1AB" stroke-width="4" stroke-linecap="round"><path d="M34 34a20 20 0 0 0 0 28M62 34a20 20 0 0 1 0 28M26 26a31 31 0 0 0 0 44M70 26a31 31 0 0 1 0 44"/></g><circle cx="48" cy="48" r="6" fill="#9AA1AB"/>`,
    clear: `<circle cx="48" cy="48" r="44" fill="#E7F5EC"/><g fill="none" stroke="#15803D" stroke-width="4" stroke-linecap="round"><path d="M34 34a20 20 0 0 0 0 28M62 34a20 20 0 0 1 0 28M26 26a31 31 0 0 0 0 44M70 26a31 31 0 0 1 0 44"/></g><circle cx="48" cy="48" r="6" fill="#15803D"/>`,
    brake: `<path d="M48 8 L90 84 H6 Z" fill="#fff" stroke="#D92D20" stroke-width="8" stroke-linejoin="round"/><rect x="44" y="34" width="8" height="28" rx="3" fill="#16191D"/><circle cx="48" cy="71" r="5" fill="#16191D"/>`,
    lost: `<circle cx="48" cy="48" r="44" fill="#FFF4DC"/><g fill="none" stroke="#B86E00" stroke-width="4" stroke-linecap="round"><path d="M34 34a20 20 0 0 0 0 28M62 34a20 20 0 0 1 0 28M26 26a31 31 0 0 0 0 44M70 26a31 31 0 0 1 0 44"/><path d="M24 72 L72 24" stroke-width="5"/></g>`
  };
  let centreState = "";
  function setCentre(state, title, sub) {
    if (state !== centreState) {
      centreState = state;
      $("cIcon").innerHTML = ICON[state === "wait" ? "standby" : state];
      $("centre").className = "card center" + (state === "brake" ? " alarm" : state === "lost" ? " lost" : "");
    }
    $("cTitle").textContent = title; $("cSub").textContent = sub;
  }

  // ---------------- UI ----------------
  function renderChips() {
    $("chips").innerHTML = ports.map(P =>
      `<span class="chip"><i style="background:${P.boardId ? COLOR[P.boardId] : "#9AA1AB"}"></i>${esc(P.demo ? P.label : P.boardId ? `${P.label} · ${MODEL[P.boardId]}` : "Identifying…")}<button data-key="${P.key}">${P.demo ? "Stop" : "Disconnect"}</button></span>`
    ).join("");
    $("idle").style.display = ports.length ? "none" : "";
  }
  $("chips").addEventListener("click", e => {
    const b = e.target.closest("button[data-key]"); if (!b) return;
    const P = ports.find(p => p.key === +b.dataset.key); if (P) disconnect(P, false);
  });

  function setIdle(title, msg) { $("idleTitle").textContent = title; $("idleMsg").textContent = msg; }

  function renderUI() {
    const t = now();
    for (const id of [1, 2]) {
      const v = V[id], card = $("card" + id), live = isLive(v), br = live && isBraking(v);
      card.classList.toggle("braking", br);
      const pill = q(id, "pill");
      pill.className = "pill " + (!v.last ? "" : !live ? "warn" : br ? "brake" : "ok");
      pill.textContent = !v.last ? "No data" : !live ? "No signal" : br ? "Braking" : "Normal";
      q(id, "peak").textContent = v.peak === null || v.peak >= 0 ? "—" : fmt(v.peak) + " m/s²";
      q(id, "brakes").textContent = v.brakes;
      q(id, "tx").textContent = v.tx.length ? Math.round(v.tx.filter(Boolean).length / v.tx.length * 100) + " %" : "—";
      q(id, "via").textContent = !v.last ? "—" : (v.via === "USB" ? "USB" : "Radio") + " · " + ago(v.last);
    }

    // radio link
    const anyPort = ports.length > 0;
    const rxLive = link.lastRx && t - link.lastRx < LIVE_MS;
    if (link.lastRx && !rxLive && !link.lostLogged && anyPort) { log("err", "No packets from the oncoming vehicle — radio link lost"); link.lostLogged = true; }
    $("rxRate").textContent = (link.rxTimes.filter(x => t - x < 5000).length / 5).toFixed(1);
    $("rxCount").textContent = link.rx.toLocaleString();
    $("txCount").textContent = `${(V[1].txOk + V[2].txOk).toLocaleString()} / ${(V[1].txFail + V[2].txFail).toLocaleString()}`;
    $("rxAge").textContent = link.lastRx ? ago(link.lastRx) : "—";
    const recent = V[1].tx.concat(V[2].tx);
    $("txPct").textContent = recent.length ? Math.round(recent.filter(Boolean).length / recent.length * 100) + " %" : "—";

    const rt = $("tagRadio");
    rt.className = "tag " + (!anyPort ? "" : rxLive ? "good" : link.lastRx ? "bad" : "warn");
    rt.querySelector("b").textContent = !anyPort ? "Offline" : rxLive ? "Live" : link.lastRx ? "Lost" : "Waiting";

    // centre display
    const braking = [1, 2].filter(id => isLive(V[id]) && isBraking(V[id]));
    if (braking.length === 2) setCentre("brake", "Both vehicles braking", "Hard braking on both sides of the bend. Slow down and keep left.");
    else if (braking.length) {
      const a = braking[0], b = a === 1 ? 2 : 1;
      setCentre("brake", "Oncoming vehicle braking", `Vehicle ${a} is braking hard beyond the bend. Vehicle ${b} has been warned to slow down.`);
    }
    else if (!anyPort) setCentre("standby", "V2V standby", "Connect a vehicle board to start.");
    else if (!isLive(V[1]) && !isLive(V[2])) setCentre("wait", "Waiting for data…", "Board connected — waiting for its first readings.");
    else if (link.lastRx && !rxLive) setCentre("lost", "Radio link lost", "The oncoming vehicle is not being heard. Check the NRF24 power and range.");
    else if (!link.lastRx) setCentre("wait", "Waiting for the other vehicle", "This board is running, but no radio packets have arrived yet.");
    else setCentre("clear", "Road clear around the bend", "Both vehicles are sharing data over the radio. No hard braking.");

    if (eventsDirty) {
      eventsDirty = false;
      $("events").innerHTML = events.length
        ? events.map(e => `<li class="${e.kind}"><time>${e.time}</time><i></i><span>${esc(e.text)}${e.n > 1 ? `<span class="x">×${e.n}</span>` : ""}</span></li>`).join("")
        : `<li class="empty">Nothing yet.</li>`;
    }
    if (rawDirty && $("console").open) {
      rawDirty = false;
      const pre = $("raw"), stick = pre.scrollTop + pre.clientHeight >= pre.scrollHeight - 30;
      pre.innerHTML = raw.map(([id, label, l]) => `<span class="${id ? "t" + id : "tx"}">[${esc(label)}]</span> ${esc(l)}`).join("\n");
      if (stick) pre.scrollTop = pre.scrollHeight;
    }
    $("clock").textContent = new Date().toTimeString().slice(0, 5);
  }
  setInterval(renderUI, 100);
  $("console").addEventListener("toggle", () => { rawDirty = true; });

  // ---------------- canvas helpers ----------------
  function fit(cv) {
    const r = cv.getBoundingClientRect(), d = window.devicePixelRatio || 1;
    const w = Math.round(r.width * d), h = Math.round(r.height * d);
    const resized = cv.width !== w || cv.height !== h;
    if (resized) { cv.width = w; cv.height = h; }
    const ctx = cv.getContext("2d"); ctx.setTransform(d, 0, 0, d, 0, 0);
    return {ctx, W: r.width, H: r.height, d, resized};
  }
  function rr(ctx, x, y, w, h, r) {
    ctx.beginPath();
    ctx.moveTo(x + r, y); ctx.arcTo(x + w, y, x + w, y + h, r); ctx.arcTo(x + w, y + h, x, y + h, r);
    ctx.arcTo(x, y + h, x, y, r); ctx.arcTo(x, y, x + w, y, r); ctx.closePath();
  }
  function inPoly(x, y, poly) {
    let c = false;
    for (let i = 0, j = poly.length - 1; i < poly.length; j = i++) {
      const a = poly[i], b = poly[j];
      if ((a[1] > y) !== (b[1] > y) && x < (b[0] - a[0]) * (y - a[1]) / (b[1] - a[1]) + a[0]) c = !c;
    }
    return c;
  }

  // ---------------- mountain road ----------------

  // Hairpin around a hill spur: Vehicle 1 drives the lower branch towards the bend,
  // Vehicle 2 comes down the upper branch — the hill hides them from each other.
  const CTRL = [[-0.06, 0.82], [0.16, 0.83], [0.38, 0.81], [0.58, 0.76], [0.73, 0.66], [0.79, 0.5],
                [0.74, 0.34], [0.59, 0.24], [0.39, 0.19], [0.17, 0.17], [-0.06, 0.17]];
  const roadCv = $("road");
  const waves = [];
  let geo = null, lastFrame = now();
  const car = {1: {p: 0.3, spd: 0}, 2: {p: 0.3, spd: 0}};

  function buildGeo(W, H, d) {
    const P = CTRL.map(([x, y]) => [x * W, y * H]), pts = [];
    for (let i = 0; i < P.length - 1; i++) {
      const p0 = P[i - 1] || P[i], p1 = P[i], p2 = P[i + 1], p3 = P[i + 2] || P[i + 1];
      for (let k = 0; k < 24; k++) {
        const t = k / 24, t2 = t * t, t3 = t2 * t;
        const f = (a, b, c, e) => 0.5 * (2 * b + (-a + c) * t + (2 * a - 5 * b + 4 * c - e) * t2 + (-a + 3 * b - 3 * c + e) * t3);
        pts.push({x: f(p0[0], p1[0], p2[0], p3[0]), y: f(p0[1], p1[1], p2[1], p3[1])});
      }
    }
    pts.push({x: P[P.length - 1][0], y: P[P.length - 1][1]});
    let len = 0;
    pts.forEach((p, i) => {
      const a = pts[Math.max(0, i - 1)], b = pts[Math.min(pts.length - 1, i + 1)];
      const dx = b.x - a.x, dy = b.y - a.y, l = Math.hypot(dx, dy) || 1;
      p.tx = dx / l; p.ty = dy / l; p.nx = p.ty; p.ny = -p.tx;      // n = inner side (towards the hill)
      if (i) len += Math.hypot(p.x - pts[i - 1].x, p.y - pts[i - 1].y);
      p.s = len;
    });
    const roadW = Math.max(30, Math.min(54, H * 0.115));
    const gap = (CTRL[0][1] - CTRL[CTRL.length - 1][1]) * H;
    const off = dist => pts.map(p => [p.x + p.nx * dist, p.y + p.ny * dist]);
    const hill = off(roadW / 2 + 5);

    // static terrain layer
    const bg = document.createElement("canvas");
    bg.width = Math.round(W * d); bg.height = Math.round(H * d);
    const c = bg.getContext("2d"); c.setTransform(d, 0, 0, d, 0, 0);
    c.fillStyle = "#EEF1EA"; c.fillRect(0, 0, W, H);
    const line = (poly, color, w, dash) => {
      c.save(); c.strokeStyle = color; c.lineWidth = w; c.lineJoin = "round"; if (dash) c.setLineDash(dash);
      c.beginPath(); poly.forEach(([x, y], i) => i ? c.lineTo(x, y) : c.moveTo(x, y)); c.stroke(); c.restore();
    };
    // valley-side contours
    for (let k = 1; k <= 7; k++) line(off(-(roadW / 2 + 14 + k * k * 4 + k * 16)), "rgba(120,140,100,.22)", 1);
    // hill: nested contour bands up to the ridge line
    const fills = ["#E2E9D8", "#D7E1CB", "#CCD9BE", "#C0CFB0", "#B4C5A2", "#A9BC96"];
    const maxD = (gap - roadW) / 2 - 4;
    for (let k = 0; k < fills.length; k++) {
      const dist = roadW / 2 + 5 + k * (maxD - roadW / 2 - 5) / fills.length;
      const poly = off(dist);
      c.fillStyle = fills[k];
      c.beginPath(); poly.forEach(([x, y], i) => i ? c.lineTo(x, y) : c.moveTo(x, y)); c.closePath(); c.fill();
      line(poly, "rgba(90,112,70,.30)", 1);
    }
    // trees on the hill (fixed pseudo-random positions)
    let seed = 7; const rnd = () => (seed = (seed * 16807) % 2147483647) / 2147483647;
    const inner = off(roadW / 2 + 16);
    for (let i = 0; i < 520; i++) {
      const x = rnd() * W, y = rnd() * H;
      if (!inPoly(x, y, inner)) continue;
      const r = 2.2 + rnd() * 2.6;
      c.fillStyle = "rgba(92,122,72,.55)"; c.beginPath(); c.arc(x + 1, y + 1, r, 0, 7); c.fill();
      c.fillStyle = "rgba(120,152,96,.85)"; c.beginPath(); c.arc(x, y, r, 0, 7); c.fill();
    }
    c.font = "600 12px Barlow, sans-serif"; c.fillStyle = "rgba(61,74,55,.75)"; c.textAlign = "center";
    c.fillText("HILL — BLOCKS THE VIEW", W * 0.36, H * 0.5 + 4);
    // road
    const road = pts.map(p => [p.x, p.y]);
    line(road, "#C9CDD2", roadW + 6);
// shoulder
    line(road, "#666B73", roadW);
// asphalt
    line(off(roadW / 2 - 4), "#F4F4F2", 1.6);
```

```
    line(off(-(roadW / 2 - 4)), "#F4F4F2",
1.6);
    line(road, "#F4F4F2", 1.6, [12, 12]);
    // guard rail on the valley side
    line(off(-(roadW / 2 + 6)), "#9AA1AB",
2.2);
    line(off(-(roadW / 2 + 6)), "#6E757F", 4,
[2, 14]);
    // arrows showing direction of travel
    const arrow = (s, side, dir) => {
      const p = at(s, pts), x = p.x + p.nx *
side, y = p.y + p.ny * side, a =
Math.atan2(p.ty * dir, p.tx * dir);
      c.save(); c.translate(x, y); c.rotate(a);
c.fillStyle = "rgba(255,255,255,.55)";
      c.beginPath(); c.moveTo(7, 0); c.lineTo(-
5, -5); c.lineTo(-2, 0); c.lineTo(-5, 5);
c.closePath(); c.fill(); c.restore();
    };
    const lane = roadW * 0.25;
    for (const f of [0.12, 0.3]) { arrow(len *
f, lane, 1); arrow(len * (1 - f), -lane, -1); }
    return {pts, len, roadW, lane, hill, bg};
  }

  function at(s, pts) {
    s = Math.max(0, Math.min(pts[pts.length -
1].s, s));
    let lo = 0, hi = pts.length - 1;
    while (hi - lo > 1) { const m = (lo + hi)
>> 1; pts[m].s < s ? lo = m : hi = m; }
    const a = pts[lo], b = pts[hi], f = (s -
a.s) / ((b.s - a.s) || 1);
    return {x: a.x + (b.x - a.x) * f, y: a.y +
(b.y - a.y) * f, tx: a.tx, ty: a.ty, nx: a.nx,
ny: a.ny};
  }

  function drawCar(ctx, x, y, a, len, wid,
color, braking, t) {
    ctx.save(); ctx.translate(x, y);
ctx.rotate(a);
    if (braking) {
      const g = ctx.createRadialGradient(-len /
2, 0, 1, -len / 2, 0, len * 0.9);
      g.addColorStop(0, "rgba(255,40,30,.55)");
g.addColorStop(1, "rgba(255,40,30,0)");
      ctx.fillStyle = g; ctx.fillRect(-len *
1.5, -len, len * 1.1, len * 2);
    }
    ctx.shadowColor = "rgba(0,0,0,.35)";
ctx.shadowBlur = 5; ctx.shadowOffsetY = 2;
    ctx.fillStyle = color; rr(ctx, -len / 2, -
wid / 2, len, wid, wid * 0.32); ctx.fill();
    ctx.shadowColor = "transparent";
    const g = ctx.createLinearGradient(0, -
wid / 2, 0, wid / 2);
    g.addColorStop(0, "rgba(0,0,0,.18)");
g.addColorStop(0.5, "rgba(255,255,255,.18)");
g.addColorStop(1, "rgba(0,0,0,.18)");
    ctx.fillStyle = g; rr(ctx, -len / 2, -wid /
2, len, wid, wid * 0.32); ctx.fill();
    // glass + roof
    ctx.fillStyle = "#1E2833";
    ctx.beginPath(); ctx.moveTo(len * 0.06, -
wid * 0.36); ctx.lineTo(len * 0.22, -wid *
0.42); ctx.lineTo(len * 0.22, wid * 0.42);
ctx.lineTo(len * 0.06, wid * 0.36);
ctx.closePath(); ctx.fill();
    ctx.beginPath(); ctx.moveTo(-len * 0.3, -
wid * 0.36); ctx.lineTo(-len * 0.22, -wid *
0.33); ctx.lineTo(-len * 0.22, wid * 0.33);
ctx.lineTo(-len * 0.3, wid * 0.36);
ctx.closePath(); ctx.fill();
    ctx.fillStyle = "rgba(255,255,255,.12)";
rr(ctx, -len * 0.21, -wid * 0.34, len * 0.26,
wid * 0.68, 3); ctx.fill();
    // mirrors
    ctx.fillStyle = color; ctx.fillRect(len *
0.08, -wid / 2 - 2.5, 3, 2.5); ctx.fillRect(len
* 0.08, wid / 2, 3, 2.5);
    // lights
    ctx.fillStyle = "#FFF8E1";
    ctx.fillRect(len / 2 - 3, -wid * 0.4, 2.5,
wid * 0.22); ctx.fillRect(len / 2 - 3, wid *
0.18, 2.5, wid * 0.22);
    ctx.fillStyle = braking ? "#FF2A1F" :
"#8E1B14";
    if (braking) { ctx.shadowColor = "#FF2A1F";
ctx.shadowBlur = 12; }
    ctx.fillRect(-len / 2 + 0.5, -wid * 0.42,
2.5, wid * 0.24); ctx.fillRect(-len / 2 + 0.5,
wid * 0.18, 2.5, wid * 0.24);
    ctx.restore();
  }

  function tagAt(ctx, x, y, text, color) {
    ctx.font = "600 12px Barlow, sans-serif";
    const w = ctx.measureText(text).width + 16;
    ctx.fillStyle = "rgba(255,255,255,.95)";
ctx.strokeStyle = "rgba(16,24,40,.12)";
ctx.lineWidth = 1;
    rr(ctx, x - w / 2, y - 11, w, 22, 6);
ctx.fill(); ctx.stroke();
    ctx.fillStyle = color; ctx.fillRect(x - w /
2 + 6, y - 4, 3, 8);
    ctx.fillStyle = "#16191D"; ctx.textAlign =
"center"; ctx.textBaseline = "middle";
ctx.fillText(text, x + 3, y + 0.5);
  }

  function warnSign(ctx, x, y, t) {
    const s = 1 + 0.07 * Math.sin(t / 110), h =
26 * s;
    ctx.save(); ctx.translate(x, y);
    ctx.shadowColor = "rgba(0,0,0,.25)";
ctx.shadowBlur = 6;
    ctx.fillStyle = "#fff"; ctx.strokeStyle =
"#D92D20"; ctx.lineWidth = 4; ctx.lineJoin =
"round";
    ctx.beginPath(); ctx.moveTo(0, -h / 2);
ctx.lineTo(h * 0.58, h / 2); ctx.lineTo(-h *
0.58, h / 2); ctx.closePath(); ctx.fill();
ctx.stroke();
    ctx.shadowColor = "transparent";
    ctx.fillStyle = "#16191D"; ctx.fillRect(-
1.5, -h * 0.18, 3, h * 0.4); ctx.beginPath();
ctx.arc(0, h * 0.32, 1.9, 0, 7); ctx.fill();
    ctx.restore();
  }

  let losState = "";
  function drawRoad() {
    const t = now(), dt = Math.min(0.05, (t -
lastFrame) / 1000); lastFrame = t;
    const {ctx, W, H, d, resized} =
fit(roadCv);
    if (W < 10 || H < 10) return;
    if (resized || !geo) geo = buildGeo(W, H,
d);
    const {pts, len, roadW, lane, hill} = geo;
    ctx.drawImage(geo.bg, 0, 0, W, H);

    const anyLive = isLive(V[1]) ||
isLive(V[2]);
    const b = {1: isLive(V[1]) &&
isBraking(V[1]), 2: isLive(V[2]) &&
isBraking(V[2])};
    for (const id of [1, 2]) {
      const c = car[id], target = anyLive ?
(b[id] ? 0.22 : 1) : 0;
      c.spd += (target - c.spd) * Math.min(1,
dt * (target < c.spd ? 4 : 1.2));
      c.p = Math.min(1.08, c.p + c.spd * dt /
14);
    }
    if (car[1].p >= 1.08 && car[2].p >= 1.08) {
car[1].p = car[2].p = -0.04; }

    const P1 = at(car[1].p * len, pts), P2 =
at((1 - car[2].p) * len, pts);
    const c1 = {x: P1.x + P1.nx * lane, y: P1.y
+ P1.ny * lane, a: Math.atan2(P1.ty, P1.tx)};
    const c2 = {x: P2.x - P2.nx * lane, y: P2.y
- P2.ny * lane, a: Math.atan2(-P2.ty, -P2.tx)};
    const on = c => c.x > -20 && c.x < W + 20;

    // line of sight between the two drivers
    let blocked = false, hit = null;
    if (on(c1) && on(c2)) {
      for (let k = 1; k < 40; k++) {
        const x = c1.x + (c2.x - c1.x) * k /
40, y = c1.y + (c2.y - c1.y) * k / 40;
        if (inPoly(x, y, hill)) { blocked =
true; if (!hit) hit = {x, y}; }
      }
      ctx.save(); ctx.setLineDash([5, 6]);
ctx.lineWidth = 1.6;
      ctx.strokeStyle = blocked ?
"rgba(217,45,32,.6)" : "rgba(21,128,61,.6)";
      ctx.beginPath(); ctx.moveTo(c1.x, c1.y);
ctx.lineTo(c2.x, c2.y); ctx.stroke();
ctx.restore();
      if (blocked) {
        ctx.save(); ctx.translate(hit.x,
hit.y); ctx.strokeStyle = "#D92D20";
ctx.lineWidth = 3; ctx.lineCap = "round";
        ctx.beginPath(); ctx.moveTo(-6, -6);
ctx.lineTo(6, 6); ctx.moveTo(6, -6);
ctx.lineTo(-6, 6); ctx.stroke(); ctx.restore();
      }
    }
    const ls = !(on(c1) && on(c2)) ? "" :
blocked ? "bad" : "good";
    if (ls !== losState) {
      losState = ls;
      const tg = $("tagLos"); tg.className =
"tag " + ls;
      tg.querySelector("b").textContent = ls
=== "bad" ? "Blocked by hill" : ls === "good" ?
"Clear" : "—";
    }

    // radio link: an arc over the hill
    const rxLive = link.lastRx && t -
link.lastRx < LIVE_MS;
    const mx = (c1.x + c2.x) / 2, my = (c1.y +
c2.y) / 2, dd = Math.hypot(c2.x - c1.x, c2.y -
c1.y) || 1;
    let px = -(c2.y - c1.y) / dd, py = (c2.x -
c1.x) / dd;
    if (px < 0) { px = -px; py = -py; }
    const cx = mx + px * Math.min(160, dd *
0.35), cy = my + py * Math.min(160, dd * 0.35);
    const bez = (u, A, B) => { const v = 1 - u;
return {x: v * v * A.x + 2 * v * u * cx + u * u
* B.x, y: v * v * A.y + 2 * v * u * cy + u * u
* B.y}; };
    if (rxLive && on(c1) && on(c2)) {
      ctx.save(); ctx.setLineDash([2, 7]);
ctx.lineDashOffset = -t / 35; ctx.lineWidth =
2; ctx.lineCap = "round";
      ctx.strokeStyle = "rgba(37,99,201,.55)";
      ctx.beginPath(); ctx.moveTo(c1.x, c1.y);
ctx.quadraticCurveTo(cx, cy, c2.x, c2.y);
ctx.stroke(); ctx.restore();
    }
    for (let i = waves.length - 1; i >= 0; i--)
{
      const w = waves[i], p = (t - w.t0) /
1000;
      if (p >= 1) { waves.splice(i, 1);
continue; }
      const S = w.from === 1 ? c1 : c2, R =
w.from === 1 ? c2 : c1;
      if (!on(S)) continue;
      const col = w.em ? "217,45,32" : (w.from
=== 1 ? "37,99,201" : "224,112,27");
      ctx.strokeStyle = `rgba(${col},${(1 - p)
* 0.55})`; ctx.lineWidth = 1.6;
      ctx.beginPath(); ctx.arc(S.x, S.y, 10 + p
* 46, 0, Math.PI * 2); ctx.stroke();
      if (on(R)) {
        const q2 = w.from === 1 ? bez(p, c1,
c2) : bez(1 - p, c1, c2);
        ctx.fillStyle = `rgba(${col},${1 - p *
0.5})`;
        ctx.beginPath(); ctx.arc(q2.x, q2.y,
w.em ? 5 : 3.5, 0, 7); ctx.fill();
      }
    }

    // cars
    const L = roadW * 0.78, Wd = roadW * 0.36;
    for (const [id, c] of [[1, c1], [2, c2]]) {
      if (!on(c)) continue;
      if (b[id]) {
        const r = 22 + 10 * ((t / 600) % 1);
        ctx.strokeStyle = `rgba(217,45,32,${0.6
- 0.5 * ((t / 600) % 1)})`; ctx.lineWidth = 3;
        ctx.beginPath(); ctx.arc(c.x, c.y, r,
0, 7); ctx.stroke();
      }
      drawCar(ctx, c.x, c.y, c.a, L, Wd,
COLOR[id], b[id], t);
    }
    for (const [id, c] of [[1, c1], [2, c2]]) {
      if (!on(c)) continue;
      const v = V[id], other = id === 1 ? 2 :
1;
      const label = `V${id}` + (v.ax === null ?
"" : `  ${fmt(v.ax)} m/s²`);
      const above = c.y > H * 0.5 ? -1 : 1;
      const ty = c.y + above * (roadW * 0.5 +
20);
      tagAt(ctx, Math.max(60, Math.min(W - 60,
c.x)), ty, label, COLOR[id]);
      if (b[other] && rxLive) warnSign(ctx,
Math.max(30, Math.min(W - 30, c.x + 70)), ty,
t);
    }
  }

  // ---------------- chart ----------------
  const chartCv = $("chart");
  function drawChart() {
    const {ctx, W, H} = fit(chartCv);
    const t = now(), span = 20000;
    const pl = 40, pr = 10, ptp = 8, pb = 24,
cw = W - pl - pr, ch = H - ptp - pb;
    let lo = -10, hi = 10;
    for (const id of [1, 2]) for (const p of
V[id].hist) { if (p.ax < lo) lo =
Math.floor(p.ax); if (p.ax > hi) hi =
Math.ceil(p.ax); }
    const Y = v => ptp + (hi - v) / (hi - lo) *
ch, X = tt => pl + (1 - (t - tt) / span) * cw;
    ctx.clearRect(0, 0, W, H);
    ctx.fillStyle = "rgba(217,45,32,.06)";
ctx.fillRect(pl, Y(TH), cw, Y(lo) - Y(TH));
    ctx.font = "500 11px 'IBM Plex Mono',
Consolas, monospace"; ctx.textBaseline =
"middle";
    const step = (hi - lo) > 24 ? 10 : 5;
    for (let v = Math.ceil(lo / step) * step; v
<= hi; v += step) {
      ctx.strokeStyle = v === 0 ? "#C3C9D1" :
"#EEF0F3"; ctx.lineWidth = 1;
      ctx.beginPath(); ctx.moveTo(pl, Y(v)
+ .5); ctx.lineTo(W - pr, Y(v) + .5);
ctx.stroke();
      ctx.fillStyle = "#8A919C"; ctx.textAlign
= "right"; ctx.fillText((v < 0 ? "−" : "") +
Math.abs(v), pl - 8, Y(v));
    }
    ctx.textAlign = "center"; ctx.textBaseline
= "alphabetic";
    for (let s = 0; s <= 20; s += 5)
ctx.fillText(s === 0 ? "now" : `-${s} s`,
Math.max(pl + 14, Math.min(W - pr - 14, pl + (1
- s / 20) * cw)), H - 6);
    ctx.save(); ctx.strokeStyle = "#D92D20";
ctx.setLineDash([5, 4]); ctx.lineWidth = 1.5;
    ctx.beginPath(); ctx.moveTo(pl, Y(TH));
ctx.lineTo(W - pr, Y(TH)); ctx.stroke();
ctx.restore();
    ctx.save(); ctx.beginPath(); ctx.rect(pl,
ptp, cw, ch); ctx.clip();
    for (const id of [1, 2]) {
      const h = V[id].hist; if (h.length < 2)
continue;
      ctx.strokeStyle = COLOR[id];
ctx.lineWidth = 2; ctx.lineJoin = "round";
      ctx.beginPath(); h.forEach((p, i) => i ?
ctx.lineTo(X(p.t), Y(p.ax)) :
ctx.moveTo(X(p.t), Y(p.ax))); ctx.stroke();
      const last = h[h.length - 1];
      if (t - last.t < LIVE_MS) { ctx.fillStyle
= COLOR[id]; ctx.beginPath();
ctx.arc(X(last.t), Y(last.ax), 4, 0, 7);
ctx.fill(); }
    }
    ctx.restore();
  }

  let chartTick = 0, gaugeFrame = now();
  function frame() {
    const t = now(), dt = Math.min(0.05, (t -
gaugeFrame) / 1000); gaugeFrame = t;
    drawRoad();
```

```
    drawGauges(dt);
    if (++chartTick % 2 === 0) drawChart();
    requestAnimationFrame(frame);
  }
  requestAnimationFrame(frame);
  if (document.fonts)
document.fonts.ready.then(() => { geo =
null; });

  // ---------------- controls ----------------
  $("btnConnect").addEventListener("click",
connect);
  $("btnDemo").addEventListener("click",
startDemo);
  $("btnSound").addEventListener("click", e =>
{
    sound = !sound;
    e.currentTarget.textContent = sound ?
"Sound on" : "Sound off";
    e.currentTarget.classList.toggle("on",
sound);
    if (sound) beep();
  });
  $("idle").addEventListener("click", e => {
    const a = e.target.closest("[data-
act]")?.dataset.act;
    if (a === "connect") connect(); else if (a
=== "demo") startDemo();
  });
  if (!("serial" in navigator)) setIdle("Open
in Chrome or Edge", "This browser can't read
USB serial. Use Google Chrome or Microsoft Edge
on a laptop/desktop — or press Run demo to
preview.");
  renderChips();
  renderUI();
})();
</script>
</body>
</html>
```